

Algoritmos estudados

Aula 1

- Naive Bayes
- Árvore de decisão

Aula2

- KNN (Falta a parte prática)
- Regressão Logística

Aula 3

- SVM
- Redes Neurais (Perceptron)

Aula 4

- Redes Neurais (MLP)
- Avaliação de algoritmo de classificação

Planejamento Aula 3

- Revisão do KNN
- Códigos do KNN em uma ou duas bases de dados

```
from sklearn.preprocessing import LabelEncoder
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

base_credit_data = pd.read_csv("C:/Users/dioni/OneDrive/Área de Trabalho/Youth Data
708.01/Materiais/Aula03/credit_data.csv")

# Separando previsores e classe
X_credit_data = base_credit_data.iloc[:,1:4].values
print(X_credit_data)

Y_credit_data = base_credit_data.iloc[:,4].values
print(Y_credit_data)

# Separar dados de treino e teste
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, Y_train, Y_test = train_test_split(X_credit_data, Y_credit_data,
test_size=0.2, random_state=0)

# importar a biblioteca
from sklearn.neighbors import KNeighborsClassifier

# Treinar o modelo KNN
knn_credit = KNeighborsClassifier(n_neighbors=5, metric='minkowski', p = 2)
knn_credit.fit(X_train, Y_train)

# Fazer previsões
previsoes = knn_credit.predict(X_test)
print(previsoes)

# Avaliar o modelo
from sklearn.metrics import accuracy_score, classification_report
acuracia = accuracy_score(Y_test, previsoes)
print(acuracia)
print(classification_report(Y_test, previsoes))

# Visualização de árvore não é aplicável para KNN, pois não é um modelo de árvore.
# Se quiser visualizar os resultados, pode usar gráficos de dispersão ou matriz de
confusão.

# Exemplo: Matriz de confusão
from sklearn.metrics import ConfusionMatrixDisplay
ConfusionMatrixDisplay.from_estimator(knn_credit, X_test, Y_test)
plt.show()

# Agora vamos aplicar a normalização dos dados e ver o que muda na acuracia
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Treinar o modelo KNN com dados normalizados
knn_credit_scaled = KNeighborsClassifier(n_neighbors=5, metric='minkowski', p=2)
knn_credit_scaled.fit(X_train_scaled, Y_train)

# Fazer previsões
previsoes_scaled = knn_credit_scaled.predict(X_test_scaled)
print(previsoes_scaled)
```

```
# Avaliar o modelo com dados normalizados
acuracia_scaled = accuracy_score(Y_test, previsoes_scaled)
print(acuracia_scaled)
print(classification_report(Y_test, previsoes_scaled))

# para printar a matriz de confusão com dados normalizados sem grafico apenas no
terminal
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(Y_test, previsoes_scaled)
print(cm)

# Aplica o mesmo modelo para o census.csv
base_census = pd.read_csv("C:/Users/dioni/OneDrive/Área de Trabalho/Youth Data
708.01/Materiais/Aula03/census.csv")
X_census = base_census.iloc[:, :-1].values
Y_census = base_census.iloc[:, -1].values

# Verificando os dados iniciais
print(base_census.head())

# aplicar o label encoder para as colunas categóricas
# LabelEncoder

labelEncoder_workclass = LabelEncoder()
labelEncoder_education = LabelEncoder()
labelEncoder_marital_status = LabelEncoder()
labelEncoder_occupation = LabelEncoder()
labelEncoder_relationship = LabelEncoder()
labelEncoder_race = LabelEncoder()
labelEncoder_sex = LabelEncoder()
labelEncoder_native_country = LabelEncoder()

X_census[:,1] = labelEncoder_workclass.fit_transform(X_census[:,1])
X_census[:,3] = labelEncoder_education.fit_transform(X_census[:,3])
X_census[:,5] = labelEncoder_marital_status.fit_transform(X_census[:,5])
X_census[:,6] = labelEncoder_occupation.fit_transform(X_census[:,6])
X_census[:,7] = labelEncoder_relationship.fit_transform(X_census[:,7])
X_census[:,8] = labelEncoder_race.fit_transform(X_census[:,8])
X_census[:,9] = labelEncoder_sex.fit_transform(X_census[:,8])
```

```
X_census[:,13] = labelEncoder_native_country.fit_transform(X_census[:,13])

print(X_census)

# Separar dados de treino e teste
X_train_census, X_test_census, Y_train_census, Y_test_census =
train_test_split(X_census, Y_census, test_size=0.2, random_state=0)

# Treinar o modelo KNN para o census
knn_census = KNeighborsClassifier(n_neighbors=5, metric='minkowski', p=2)
knn_census.fit(X_train_census, Y_train_census)

# Fazer previsões
previsoes_census = knn_census.predict(X_test_census)
print(previsoes_census)

# Avaliar o modelo para o census
acuracia_census = accuracy_score(Y_test_census, previsoes_census)
print(acuracia_census)
print(classification_report(Y_test_census, previsoes_census))

# Visualização de matriz de confusão para o census sem grafico
cm_census = confusion_matrix(Y_test_census, previsoes_census)
print(cm_census)

# aplicar a normalização dos dados e ver o que muda na acuracia
scaler_census = StandardScaler()
X_train_census_scaled = scaler_census.fit_transform(X_train_census)
X_test_census_scaled = scaler_census.transform(X_test_census)

# Treinar o modelo KNN com dados normalizados para o census
knn_census_scaled = KNeighborsClassifier(n_neighbors=5, metric='minkowski', p=2)
knn_census_scaled.fit(X_train_census_scaled, Y_train_census)

# Fazer previsões com dados normalizados
previsoes_census_scaled = knn_census_scaled.predict(X_test_census_scaled)
print(previsoes_census_scaled)

# Avaliar o modelo com dados normalizados para o census
acuracia_census_scaled = accuracy_score(Y_test_census, previsoes_census_scaled)
print(acuracia_census_scaled)
print(classification_report(Y_test_census, previsoes_census_scaled))

# Matriz de confusão com dados normalizados para o census sem grafico
cm_census_scaled = confusion_matrix(Y_test_census, previsoes_census_scaled)
print(cm_census_scaled)
```

- Teoria da Regressão Logística
- Prática regressão Logística para classificação

```
from sklearn.linear_model import LogisticRegression

from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

base_credit_data = pd.read_csv("C:/Users/dioni/OneDrive/Área de Trabalho/Youth Data 708.01/Analise de dados 2/Aula3/credit_data.csv")

X_credit_data = base_credit_data.iloc[:,1:4].values
print(X_credit_data)

# Normalizar os dados de entrada
from sklearn.preprocessing import MinMaxScaler
normalizacao = MinMaxScaler()
X_credit_data_normalized = normalizacao.fit_transform(X_credit_data)

Y_credit_data = base_credit_data.iloc[:,4].values
print(Y_credit_data)

# Separar dados de treino e teste
from sklearn.model_selection import train_test_split

X_train, X_test, Y_train, Y_test =
train_test_split(X_credit_data_normalized, Y_credit_data,
test_size=0.2, random_state=0)

# Treinamento do modelo
# os principais do logistic regression são o solver e C
# solver: 'newton-cg', 'lbfgs', 'liblinear', 'sag', 'saga'
# C: parâmetro de regularização
# C controla o trade-off entre maximizar a margem e minimizar o erro de classificação
logistic_credit = LogisticRegression(random_state=1, solver='lbfgs',
C=1.0)
```

```

logistic_credit.fit(X_train, Y_train)

print(logistic_credit.intercept_)
print(logistic_credit.coef_)
previsoes = logistic_credit.predict(X_test)
print(previsoes)

from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
print(accuracy_score(Y_test, previsoes))
print(classification_report(Y_test, previsoes))
print(f'Matriz de confusão: \n {confusion_matrix(Y_test, previsoes)}')

```

- Teoria e prática SVM

```

from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

base_credit_data = pd.read_csv("C:/Users/dioni/OneDrive/Área de
Trabalho/Youth Data 708.01/Análise de dados 2/Aula3/credit_data.csv")

X_credit_data = base_credit_data.iloc[:,1:4].values
print(X_credit_data)

# Normalizar os dados de entrada
from sklearn.preprocessing import MinMaxScaler
normalizacao = MinMaxScaler()
X_credit_data_normalized = normalizacao.fit_transform(X_credit_data)

Y_credit_data = base_credit_data.iloc[:,4].values
print(Y_credit_data)

# Separar dados de treino e teste
from sklearn.model_selection import train_test_split

```

```
X_train, X_test, Y_train, Y_test =  
train_test_split(X_credit_data_normalized, Y_credit_data,  
test_size=0.2, random_state=0)  
  
from sklearn.svm import SVC  
  
# Treinamento do modelo  
# os principais parâmetros são o kernel e C  
# kernel: 'linear', 'poly', 'rbf', 'sigmoid'  
# C: parâmetro de regularização  
# C controla o trade-off entre maximizar a margem e minimizar o erro  
# de classificação  
  
svm_credit = SVC(kernel='rbf', random_state=1, C=2.0)  
svm_credit.fit(X_train, Y_train)  
previsoes = svm_credit.predict(X_test)  
  
from sklearn.metrics import accuracy_score, classification_report,  
confusion_matrix  
print(accuracy_score(Y_test, previsoes))  
print(classification_report(Y_test, previsoes))  
print(f'Matriz de confusão: \n {confusion_matrix(Y_test, previsoes)}')
```